



Revision Course

Course : MLAI

Live Session On : Revision Course

Instructor : Tanisha Medewala

Linear Regression models the relationship between **independent variables (features)** and a **continuous dependent variable** by fitting a linear equation. The goal is not just prediction, but also **understanding how each feature influences the target**.

Key intuition:

- Changes in input cause **proportional changes** in output
- Relationship is assumed to be **additive and linear**

This makes Linear Regression both a **predictive model** and an **explanatory model**.

2. Model Representation (Hypothesis Function)

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Interpretation:

- β_0 (Intercept): Expected value of y when all inputs are zero
- β_i : Change in output for **one-unit increase** in x_i , holding other variables constant

This interpretability is a **major advantage** over many black-box models.

3. Objective Function (Why MSE?)

Linear Regression minimizes **Mean Squared Error (MSE)**:

Why MSE is used:

1. Penalizes large errors more heavily
2. Differentiable → easy optimization
3. Convex loss → **global minimum guaranteed**

This is critical in exams:

👉 **No local minima problem in standard Linear Regression**

4. Training the Model

4.1 Normal Equation (Analytical Solution)

$$\theta = (X^T X)^{-1} X^T y$$

Pros:

- Exact solution
- No learning rate tuning

Cons:

- Computationally expensive
- Matrix inversion unstable if features are correlated

Used mostly when:

- Dataset is small
- Feature count is limited

4.2 Gradient Descent (Iterative Optimization)

Weights are updated iteratively:

$$\theta := \theta - \alpha \nabla J(\theta)$$

Pros:

- Scales to large datasets
- Works with streaming data

Cons:

- Requires learning rate tuning
- Can converge slowly

Used in real-world large-scale systems (including AWS ML pipelines).

5. Key Assumptions (Very Important)

Linear Regression relies on **statistical assumptions**. Violating them leads to **misleading coefficients**.

5.1 Linearity

The relationship between inputs and output must be linear.

✗ If violated → model underfits

✓ Fix: feature engineering (polynomial terms, transformations)

5.2 Independence of Errors

Residuals must be independent.

✗ Violation occurs in time-series or grouped data

✓ Fix: time-series models or clustered standard errors

5.3 Homoscedasticity

Variance of errors should be constant across all input values.

✗ Heteroscedasticity → unreliable confidence intervals

✓ Fix: transformations (log, sqrt)

5.4 Normality of Errors

Errors should follow a normal distribution (mainly for inference).

! Important for hypothesis testing, less critical for prediction

5.5 Low Multicollinearity

Features should not be highly correlated.

✗ High multicollinearity:

• Inflates variance of coefficients

• Makes coefficients unstable

✓ Fix: Ridge Regression, feature removal, PCA

6. Evaluation Metrics (Why They Matter)

6.1 Mean Squared Error (MSE)

• Measures average squared prediction error

• Sensitive to outliers

6.2 Root Mean Squared Error (RMSE)

• Same unit as target

• Easier to interpret than MSE

6.3 R² Score

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

• Indicates **variance explained by the model**

• R² close to 1 → good fit

• Negative R² → model worse than mean prediction

⚠ High R² does NOT guarantee good generalization.

Assumptions of Linear Regression

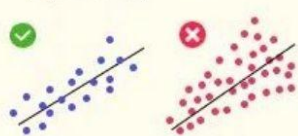
1. Linearity

(Linear relationship between Y and each X)



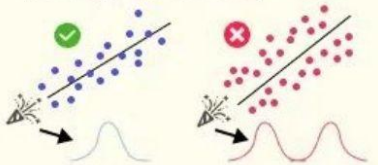
2. Homoscedasticity

(Equal variance)



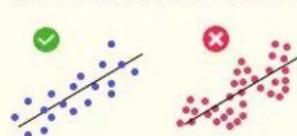
3. Multivariate Normality

(Normality of error distribution)



4. Independence

(of observations. Includes "no autocorrelation")



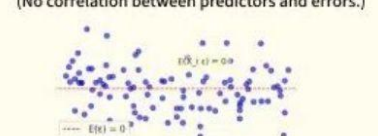
5. Lack of Multicollinearity

(Predictors are not correlated with each other)

$$X_1 + X_2 \quad X_1 \sim X_2$$

6. Absence of endogeneity

(No correlation between predictors and errors.)



1. What is Logistic Regression?

Logistic Regression is a **supervised learning algorithm** used for **classification**, primarily **binary classification**.

Despite its name, it is a **classification model**, not a regression model. It models the **probability** that an input belongs to a particular class.

2. Model Representation

Logistic Regression models the **log-odds** (logit) as a linear function of inputs.

$$z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$
$$P(y = 1 | X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- $\sigma(z)$ is the **sigmoid function**
- Output is a probability between **0 and 1**

3. Decision Boundary

- A threshold (commonly **0.5**) converts probability into class label
 - The decision boundary is **linear in feature space**
- Changing the threshold:
- Affects **precision–recall tradeoff**
 - Does NOT change model parameters

4. Loss Function (Log Loss / Cross-Entropy)

Logistic Regression minimizes **Log Loss**:

Why log loss:

- Strongly penalizes confident wrong predictions
- Convex → guarantees **global minimum**

$$J = -\frac{1}{n} \sum [y \log(p) + (1 - y) \log(1 - p)]$$

5. Training Method

- Trained using **Gradient Descent** or its variants
- No closed-form solution like linear regression
- Uses **maximum likelihood estimation (MLE)**

6. Interpretation of Coefficients

- Coefficients represent change in **log-odds** per unit increase in feature
 - Sign of coefficient:
 - Positive → increases probability of class 1
 - Negative → decreases probability of class 1
- Exponentiating coefficients gives **odds ratios**.

7. Assumptions

Logistic Regression assumes:

1. **Linear relationship between features and log-odds**
2. **Independent observations**
3. **Little multicollinearity**
4. **Large sample size** (important for stable estimates)

⚠ Does NOT assume normality or homoscedasticity.

Multi-Class Logistic Regression

Uses **Softmax** instead of sigmoid

Known as **Multinomial Logistic Regression**

$$P(y = k) = \frac{e^{z_k}}{\sum e^{z_j}}$$

1. Confusion Matrix (Foundation)

A **confusion matrix** summarizes prediction outcomes by comparing **actual vs predicted** classes.

For **binary classification**:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Interpretation:

- **TP**: Correctly predicted positives
- **TN**: Correctly predicted negatives
- **FP**: Incorrectly predicted positive (Type I error)
- **FN**: Missed positive cases (Type II error)

All classification metrics are derived from these four values.

2. Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

What it measures:

- Of all **predicted positives**, how many were actually correct?

When precision matters:

- Cost of **false positives** is high

Examples:

- Spam detection (important emails marked spam)
- Fraud alerts that inconvenience users

High precision → fewer false alarms.

3. Recall / Sensitivity / True Positive Rate (TPR)

$$\text{Recall} = \text{Sensitivity} = \text{TPR} = \frac{TP}{TP + FN}$$

What it measures:

- Of all **actual positives**, how many were correctly identified?

When recall matters:

- Cost of **false negatives** is high

Examples:

- Disease detection
 - Fraud detection (missing fraud is costly)
- High recall → fewer missed positives.

4. Specificity / True Negative Rate (TNR)

$$\text{Specificity} = \frac{TN}{TN + FP}$$

What it measures:

- Of all **actual negatives**, how many were correctly classified?

When specificity matters:

- Need to confidently rule out negatives

Example:

- Medical screening to avoid unnecessary treatment

5. False Positive Rate (FPR)

$$\text{FPR} = \frac{FP}{FP + TN}$$

Key relationship:

$$\text{FPR} = 1 - \text{Specificity}$$

Interpretation:

- Probability that a **negative sample** is incorrectly labeled **positive**

Important in:

- ROC curves
- Risk-sensitive decision systems

6. Accuracy (Context Only)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Limitation:

- Misleading for **imbalanced datasets**

Example:

- 99% accuracy by predicting all samples as negative

⚠️ Rarely sufficient alone.

7. Precision–Recall Tradeoff

- Increasing **recall** often lowers **precision**
- Increasing **precision** often lowers **recall**

Controlled by:

- Classification **threshold**

Lower threshold → higher recall, lower precision

Higher threshold → higher precision, lower recall

8. ROC Curve (Receiver Operating Characteristic)

The ROC curve plots:

- **X-axis:** False Positive Rate (FPR)
- **Y-axis:** True Positive Rate (TPR / Recall)

Each point corresponds to a **classification threshold**.

Interpretation:

- Measures ability to distinguish between classes
- Threshold-independent view of model performance

ROC (Receiver Operating Characteristic) is a curve that shows how a binary classifier's performance changes as the **decision threshold** varies.

It plots:

- **X-axis:** False Positive Rate (FPR)
- **Y-axis:** True Positive Rate (TPR / Recall / Sensitivity)

Each point on the curve represents a **different classification threshold**.

9. AUC-ROC Score

AUC (Area Under the ROC Curve) summarizes ROC performance in a single number.

Value range:

- **0.5** → random classifier
- **1.0** → perfect classifier

Meaning:

- Probability that the model ranks a random positive sample higher than a random negative one

10. When AUC-ROC Is Useful

- ✓ Balanced datasets
- ✓ Comparing classifiers
- ✓ Threshold-independent evaluation

11. When AUC-ROC Is Misleading

- ✗ Highly imbalanced datasets
- ✗ When positive class is rare

In such cases, **Precision–Recall curves** are preferred.

ROC focuses on **class separability**, not accuracy.

- TPR shows **benefit** (correct detection)
- FPR shows **cost** (false alarms)

This makes ROC suitable for evaluating models across **all thresholds**, not just one.

Ridge and Lasso Regression

Ridge Regression (L2)

- Adds **L2 penalty** (sum of squared coefficients)
- **Shrinks coefficients** but **never makes them zero**
- Reduces **overfitting** and handles **multicollinearity**
- Keeps **all features**
- Best when **all features are important**

Loss:

$$\text{MSE} + \lambda \sum w^2$$

Lasso Regression (L1)

- Adds **L1 penalty** (sum of absolute coefficients)
- Shrinks coefficients and **sets some to zero**
- Performs **automatic feature selection**
- Produces **sparse, interpretable models**
- Best when **many irrelevant features exist**

Loss:

$$\text{MSE} + \lambda \sum |w|$$

Aspect

Penalty

Feature selection

Multicollinearity

Sparsity

Ridge

L2



Excellent



Lasso

L1



Moderate



Use Ridge Regression (L2) when:

- **Features are highly correlated** (multicollinearity)
- You believe **most or all features are important**
- You want **stable coefficients**
- Prediction accuracy is more important than interpretability
- Dataset has **many numeric features**
👉 Ridge **shrinks coefficients** but keeps all variables.

Use Lasso Regression (L1) when:

- Dataset has **many irrelevant or redundant features**
- You want **automatic feature selection**
- Model **interpretability** is important
- High-dimensional data (e.g., text, genomics)
👉 Lasso **shrinks coefficients and sets some to zero.**

A **Decision Tree** is a supervised machine learning algorithm used for **classification and regression** that makes decisions by **recursively splitting data** based on feature values.
It mimics **human decision-making** using a tree-like structure.

1. Impurity Measures (How splits are chosen)

◆ Gini Index

$$\text{Gini} = 1 - \sum p_i^2$$

- Measures **node impurity**
- Lower Gini = purer node
- Faster to compute than entropy
- Used by **CART**
- 👉 **0 = pure node**

Entropy

$$\text{Entropy} = - \sum p_i \log_2 p_i$$

- Measures **uncertainty / randomness**
- Higher entropy = more mixed classes
- Used in **ID3 / C4.5**
- 👉 **0 = pure node**

◆ Classification Error

$$\text{Error} = 1 - \max(p_i)$$

- Simplest impurity measure
- Less sensitive to class distribution
- Rarely used for splitting

Overfitting in Decision Trees

- Trees can grow **too deep**
- Fit noise in training data
- Low bias, **high variance**
- 👉 Pruning controls overfitting.

Structure of a Decision Tree

- **Root Node** → Entire dataset
- **Decision Nodes** → Feature-based splits
- **Branches** → Outcomes of decisions
- **Leaf Nodes** → Final prediction (class or value)

Information Gain

$$IG = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$$

- Chooses split that **reduces entropy the most**

Measure

Sensitive

Used For

Gini

Yes

Faster splits

Entropy

High

Info gain

Class Error

Low

Pruning only

Pruning (Reducing Tree Complexity)

◆ Pre-Pruning (Early Stopping)

Stops tree growth early using:

- Max depth
- Min samples per node
- Min impurity decrease
- ✓ Faster
- ✗ Risk of underfitting

◆ Post-Pruning (Most Important)

- Grow full tree first
- Then **remove branches** that don't improve validation performance
- Based on:
 - Validation error
 - Cost-complexity pruning
- ✓ Better generalization
- ✓ Reduces overfitting

Decision Trees for Regression

- Split to minimize **variance / MSE**
- Leaf node predicts **mean value**

Advantages of Decision Trees

- ✓ Easy to understand & interpret
- ✓ Handles numerical & categorical data
- ✓ No feature scaling required
- ✓ Non-linear relationships

✗ Disadvantages

- ✗ Prone to overfitting
- ✗ Unstable (small data changes → different tree)
- ✗ Less accurate alone than ensembles

What Is Clustering

- **Unsupervised learning** technique
- Groups data based on **similarity**
- No labeled target variable
- Used for segmentation, pattern discovery, and exploratory analysis

2. Hierarchical Clustering

Definition

Hierarchical clustering builds a **nested hierarchy of clusters**, represented as a **dendrogram**.

Types

- **Agglomerative (bottom-up)** – most common
- **Divisive (top-down)**

Linkage Methods

- **Single:** minimum distance
- **Complete:** maximum distance
- **Average:** mean distance
- **Ward's:** minimizes variance

Key Characteristics

- No need to specify K initially
- Distance-based
- Produces hierarchical structure

Advantages

- Highly interpretable
- Reveals cluster relationships
- Useful for exploratory analysis

Limitations

- Computationally expensive ($O(n^2)$)
- Not scalable
- Sensitive to noise and outliers

When to Use

- Small datasets
- Unknown number of clusters
- Need for interpretability

1. K-Means Clustering

Definition

K-Means partitions data into **K non-overlapping clusters** by minimizing **within-cluster variance**.

Objective Function

$$\min \sum_K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Algorithm Steps

1. Choose number of clusters **K**
2. Initialize centroids
3. Assign points to nearest centroid
4. Update centroids
5. Repeat until convergence

Key Characteristics

- Distance-based (usually Euclidean)
- Produces **flat clustering**
- Requires feature scaling

Advantages

- Fast and scalable
- Simple to implement
- Efficient for large datasets

Limitations

- Must predefine K
- Sensitive to outliers and initialization
- Assumes spherical, equal-sized clusters

When to Use

- Large datasets
- Known number of clusters
- Well-separated numeric features

1. Classical NLP Overview

Classical NLP focuses on **rule-based and statistical techniques** to process and understand text **before deep learning models**.

Main levels:

- **Lexical processing** → word-level operations
- **Syntactic processing** → sentence structure and grammar

2. Lexical Processing (Word-Level Analysis)

Lexical processing deals with **individual words, symbols, and their basic properties**.

♦ Tokenization

Definition:

Breaking text into smaller units called **tokens** (words, punctuation, numbers).

Example:

“NLP is fun!” → [NLP, is, fun, !]

Types:

- Word tokenization
- Sentence tokenization
- Character tokenization

Challenges:

- Contractions (don’t)
- Hyphenated words
- URLs, emojis, abbreviations

Why important:

- First step of almost all NLP pipelines
- Errors propagate to downstream tasks

♦ Regular Expressions (Regex)

Definition:

Pattern-matching rules used to find, extract, or replace text.

Uses in NLP:

- Text cleaning
- Removing HTML tags
- Extracting emails, dates, phone numbers
- Rule-based tokenization

Strengths:

- Fast and precise
- No training required

Limitations:

- Hard to scale
- Language-dependent
- Breaks easily on unseen patterns

♦ Part-of-Speech (POS) Tagging

Definition:

Assigns grammatical labels to each token.

Example:

“Dogs bark loudly”

- Dogs → Noun
- bark → Verb
- loudly → Adverb

Approaches:

- Rule-based
- Statistical (HMM)
- Conditional Random Fields (CRF)

Why important:

- Foundation for parsing
- Helps disambiguate word meaning
- Used in NER, grammar checking

♦ Pointwise Mutual Information (PMI)

Definition:

Measures how strongly two words are associated.

$$PMI(w_1, w_2) = \log \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

Interpretation:

- High PMI → words co-occur more than chance
- Low/negative PMI → weak or no association

Uses:

- Collocation detection
- Phrase mining
- Lexical semantics

Limitations:

- Biased toward rare words
- Sensitive to data sparsity

3. Syntactic Processing (Sentence-Level Analysis)

Syntactic processing studies **how words are arranged to form sentences**.

◆ Constituency Parsing (Phrase Structure Parsing)

Definition:
Represents sentences as **nested phrases** using grammar rules.

Example structure:

```
[S
  [NP The cat]
  [VP sat on
    [NP the mat]
  ]
]
```

Grammar Used:

- Context-Free Grammar (CFG)

Advantages:

- Linguistically rich
- Explicit phrase boundaries

Limitations:

- Computationally expensive
- Less flexible for real-world text

◆ Dependency Parsing

Definition:
Models grammatical relationships between **words directly**.

Example:

- sat → cat (subject)
- sat → mat (object)

Key Idea:

- Every word depends on a **head**
- Root word governs the sentence

Advantages:

- Compact representation
- Captures syntactic relations clearly
- Widely used in modern NLP

Aspect	Constituency Parsing	Dependency Parsing
Focus	Phrase structure	Word relations
Representation	Tree of phrases	Directed graph
Grammar	CFG	Dependency grammar
Interpretability	High	High
Practical usage	Lower	Higher

Level	Focus	Techniques
Lexical	Word form & meaning	Tokenization, Regex, POS, PMI
Syntactic	Sentence structure	Constituency & Dependency parsing

1. Neuron (Artificial Neuron)

What Is a Neuron?

A neuron is the **basic computational unit** of a neural network.

How It Works

$$z = \sum w_i x_i + b$$

$$\text{output} = f(z)$$

- Inputs $\rightarrow$ weighted sum $\rightarrow$ bias $\rightarrow$ activation function
- Mimics biological neurons

Role

- Learns patterns from data
- Activation introduces non-linearity

2. Artificial Neural Network (ANN)

Definition

ANN is a network of interconnected neurons organized into layers.

Architecture

- **Input layer**
- **Hidden layer(s)**
- **Output layer**

Key Characteristics

- Fully connected layers
- Learns non-linear relationships
- Uses backpropagation for training

Limitations

- High number of parameters
- Does not exploit spatial structure
- Prone to overfitting

3. Convolutional Neural Network (CNN)

Definition

CNN is a neural network specialized for **spatial data** (images).

Core Components

- **Convolution layer** $\rightarrow$ feature extraction
- **Filters/Kernels** $\rightarrow$ detect patterns
- **Pooling layer** $\rightarrow$ downsampling
- **Fully connected layer** $\rightarrow$ classification

Key Properties

- Local connectivity
- Parameter sharing
- Translation invariance

Advantages

- Fewer parameters than ANN
- Captures spatial hierarchy
- High performance on images

4. Recurrent Neural Network (RNN)

Definition

RNN is designed for **sequential data** where order matters.

Key Idea

- Maintains a **hidden state** (memory)
- Output depends on current input and previous state

$$h_t = f(x_t, h_{t-1})$$

Applications

- Text processing
- Time-series forecasting
- Speech recognition

Limitations

- Vanishing/exploding gradients
- Poor long-term memory

